Presentación y preguntas Examen: UT3 Pruebas y Depuración en el desarrollo de software

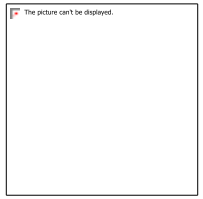
Vázquez García, Iván
Echeverría Gómez, Fernando

1. Introducción

- ¿Qué son las pruebas de software?
- ¿Por qué son fundamentales en el desarrollo de aplicaciones?
- Relación entre calidad del software y pruebas.

1.

Qué son las pruebas de software



Es el proceso técnico de evaluación del software para identificar diferencias entre lo esperado y la realidad conocidas como defectos.
Se compone de la Verificación que comprueba si cumplimos los requisitos técnicos y la Validación que asegura que el producto satisface las necesidades del usuario. Busca reducir riesgos y aportar confianza sobre el producto.

Por qué son fundamentales

Son esenciales debido a que el desarrollo es complejo y propenso a errores humanos. A nivel económico corregir un fallo en fases iniciales es mucho más rentable que hacerlo en el producto final.
Garantizan la seguridad y fiabilidad evitando pérdidas de datos o problemas críticos.

Relación calidad y pruebas.

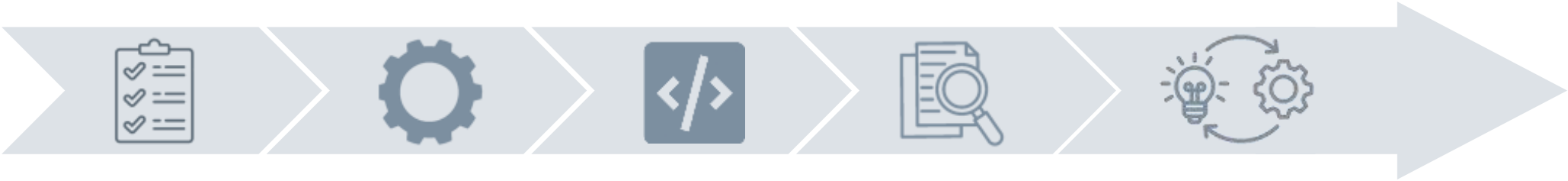
Existe una relación directa donde la calidad es el objetivo y las pruebas el instrumento para medirla. Sin un proceso de pruebas estructurado la calidad del software no se puede asegurar de forma profesional.

2. Pruebas en el proceso de desarrollo de software

- Planificación de pruebas
- Tipos de pruebas
- Procedimientos y casos de prueba

2.

Planificación de Pruebas en el Ciclo de Vida



Requisitos **Diseño** **Codificación** **Pruebas** **Implementación**

Planificación Continua

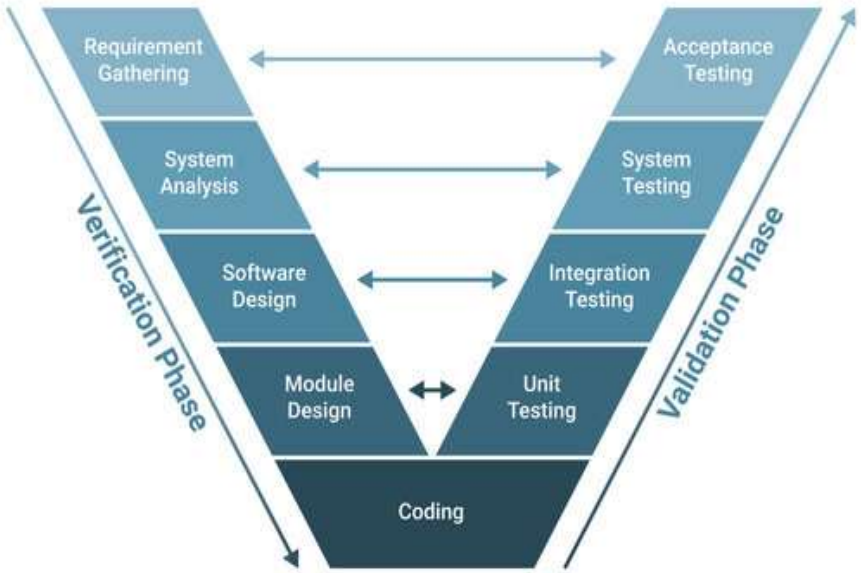
La planificación de pruebas no es una fase final aislada sino una actividad continua que comienza desde la toma de requisitos iniciales. Su objetivo es definir anticipadamente qué se probará, cómo se probará quién lo hará y cuándo se ejecutará cada tipo de prueba.

Beneficios de Planificar

Una planificación temprana ayuda a identificar riesgos técnicos y de negocio desde el principio. Permite definir el entorno de pruebas necesario con antelación evitando cuellos de botella y prisas al final del proyecto.

2.

Relación Entre Fases de Desarrollo y Pruebas



Correspondencia Directa

Existe una relación directa entre cada fase de construcción del software y un nivel de pruebas específico

Niveles de Prueba

Durante la fase de requisitos se definen las pruebas de aceptación que validará el cliente. En la fase de diseño se planifican las pruebas de sistema e integración. Durante la codificación el desarrollador crea y ejecuta las pruebas.

2.

Tipos de Pruebas. Funcionales y Estructurales

Pruebas Funcionales

Se centran exclusivamente en verificar qué hace el sistema sin importar cómo está construido por dentro. Validan que las funciones del software cumplan exactamente con los requisitos.

Pruebas Estructurales

Analizan la implementación interna del software y su arquitectura. Se enfocan en la cobertura del código asegurando que se ejecuten todas las ramas sentencias y caminos lógicos posibles dentro del programa.

2.

Tipos de Pruebas. Regresión y Enfoques de Caja

Pruebas de Regresión

Consisten en volver a probar el software tras una modificación o corrección de errores. Su objetivo crítico es asegurar que los nuevos cambios no han introducido defectos no intencionados en funcionalidades que antes funcionaban correctamente.

Caja Negra

Las pruebas de caja negra se realizan sin conocimiento del código fuente basándose únicamente en las entradas y salidas esperadas.

Caja Blanca

Las pruebas de caja blanca requieren acceso total al código fuente para examinar la lógica interna

2.

Comparación de Enfoques

Características	Caja Negra	Caja Blanca
Visión del Código	Caja Negra el código es invisible para el probador.	Caja Blanca el código es totalmente visible y analizado.
Quién Realiza la Prueba	Caja Negra suele ser realizada por testers especializados o usuarios finales.	Caja Blanca es realizada principalmente por los desarrolladores de software.
Objetivo Principal	Caja Negra busca validar requisitos y funcionalidades desde fuera.	Caja Blanca busca verificar la lógica interna los bucles y las condiciones.
Nivel de Aplicación	Caja Negra aplica a pruebas funcionales de sistema y aceptación.	Caja Blanca aplica a pruebas unitarias y de integración de bajo nivel.

2.

Procedimientos y Casos de Prueba

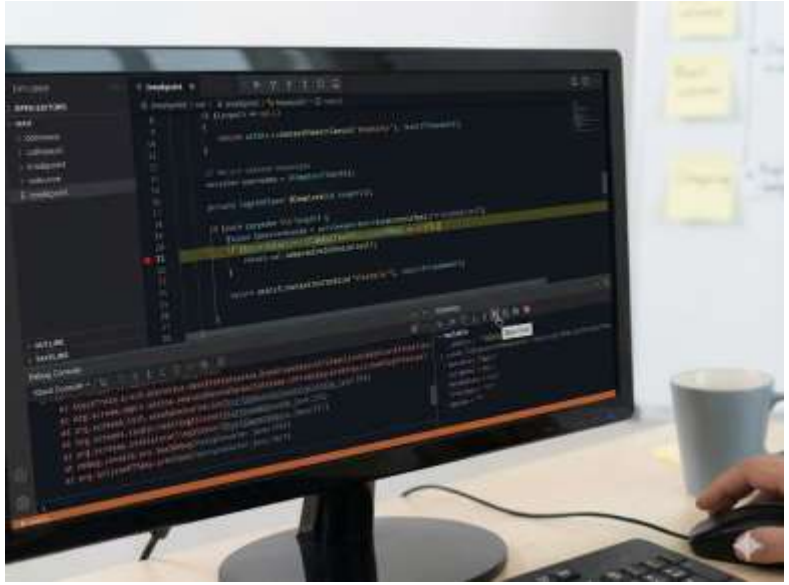
Definiciones Clave

Un **Procedimiento de Prueba** es la guía detallada que describe a secuencia de pasos para configurar el entorno ejecutar un conjunto de pruebas y registrar los resultados.

Un **Caso de Prueba** es la unidad más pequeña de prueba diseñada para verificar un único requisito o escenario específico bajo condiciones concretas

Elementos de un Caso de Prueba

Se compone de tres partes fundamentales. Primero la Entrada que son los datos o acciones suministradas al sistema. Segundo la Salida Esperada que es el resultado exacto que debería dar el sistema según las especificaciones. Tercero el Resultado Real que es lo que el sistema devuelve realmente tras la ejecución y que se compara con la esperada



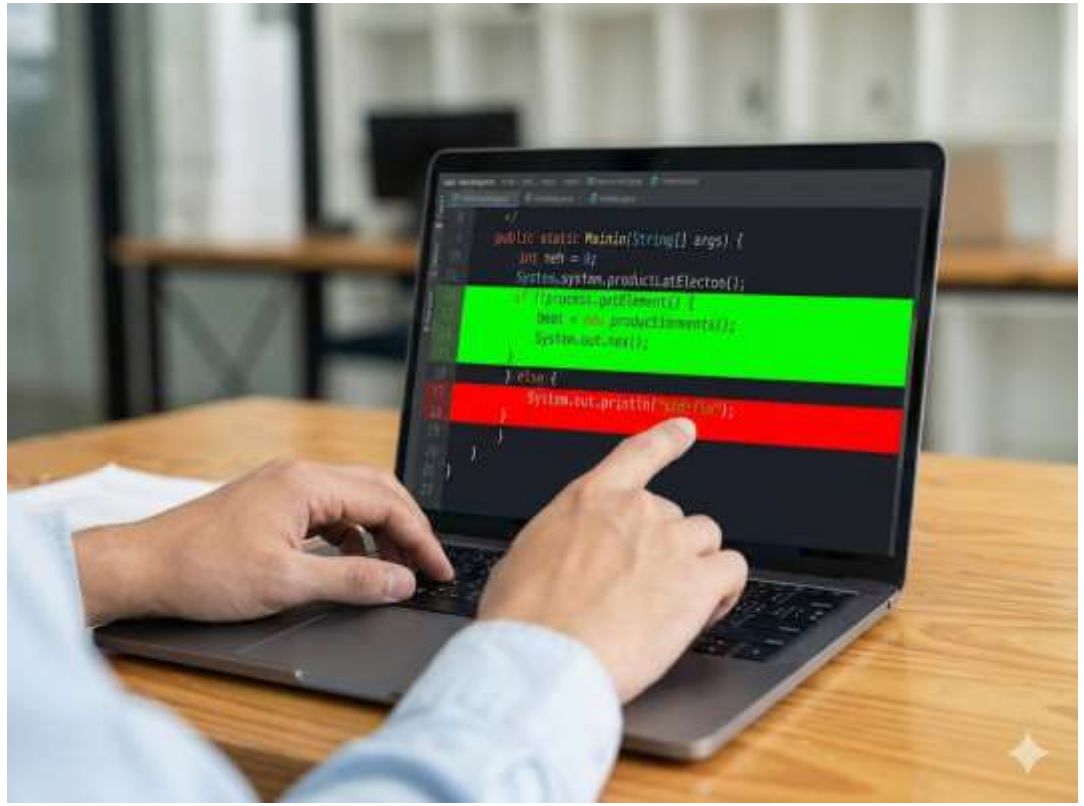
3. Pruebas de código

- Conceptos Clave
- Pruebas Unitarias
- Herramientas

3. Cobertura de Código

Cobertura de Código

La cobertura de código es una métrica técnica indispensable para calcular qué porcentaje exacto de software se ha ejecutado realmente durante las pruebas. Su objetivo fundamental es identificar visualmente las partes del programa que no han sido verificadas y podrían contener errores ocultos garantizando así que no queden zonas sin testear.



3.

Clases de Equivalencia

Clases de Equivalencia

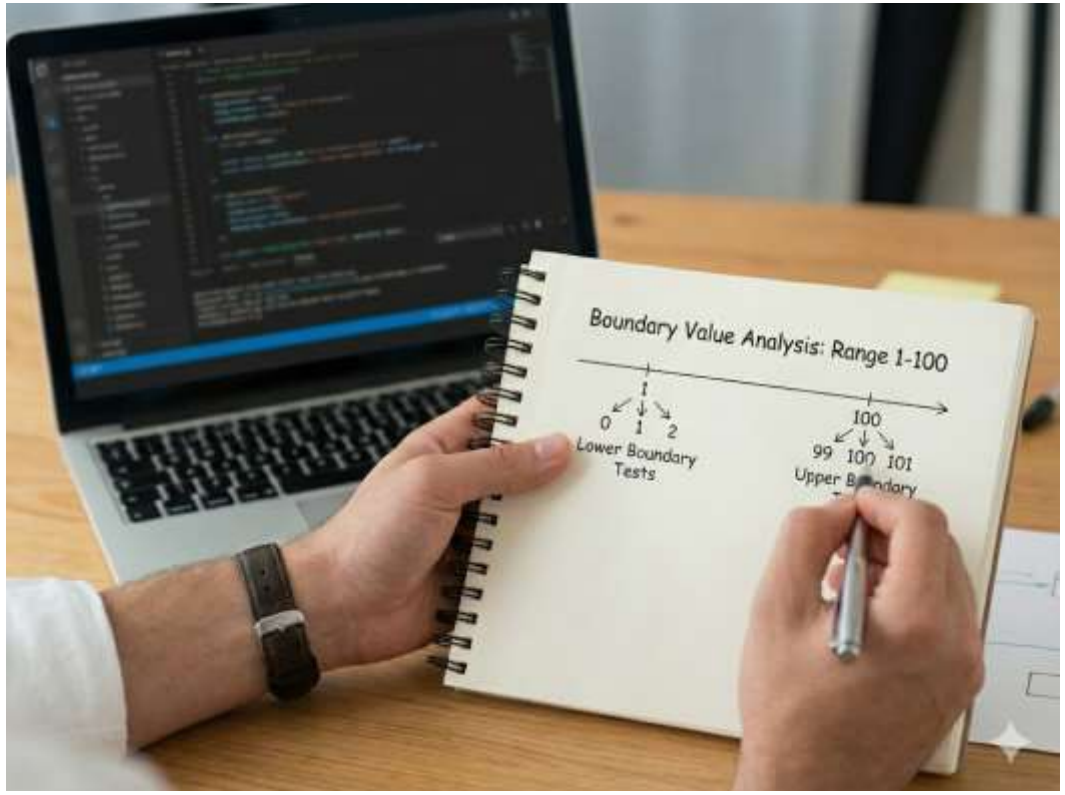
Las clases de equivalencia son una técnica eficiente que agrupa los datos de entrada con un comportamiento similar para optimizar el proceso de prueba. Esta metodología permite probar un único valor representativo de cada grupo asumiendo que el resto funcionará igual y ahorrando así una gran cantidad de tiempo y recursos sin perder calidad.

3.

Valores Límite

Valores Límite

El análisis de valores límite es una técnica crítica que se centra exclusivamente en verificar los extremos exactos o fronteras de los rangos de datos permitidos. Se utiliza obligatoriamente porque la experiencia en ingeniería demuestra que la inmensa mayoría de los fallos lógicos de programación ocurren justo en los bordes de las condiciones y no en los valores centrales.



3.

Automatización de pruebas unitarias.

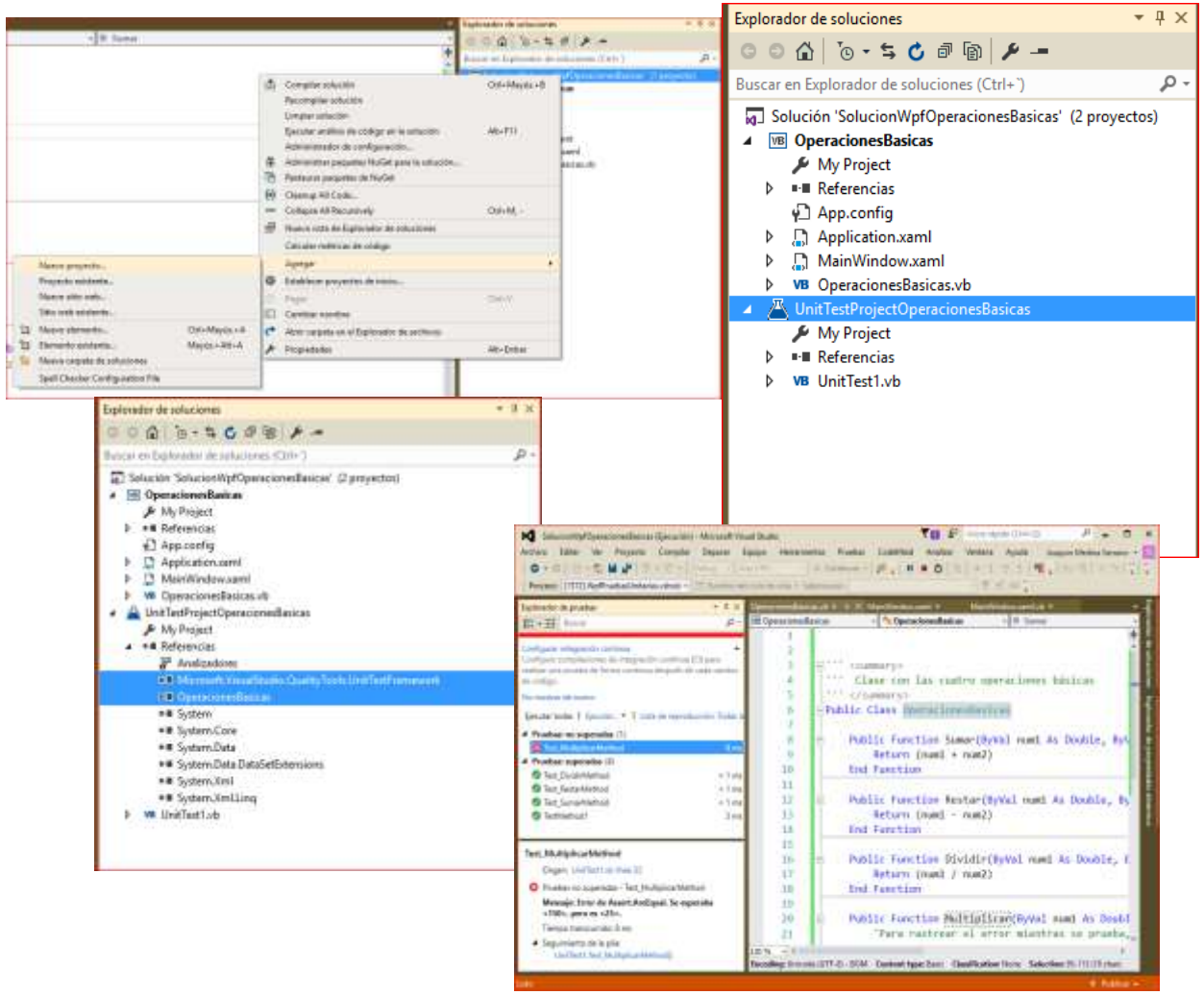
Crear scripts o programas que verifican automáticamente que cada unidad de código (función, método o clase) se comporte como se espera. Estas pruebas se ejecutan sin intervención humana cada vez que hay un cambio en el código.

Ejmplos: **JUnit** (Java), **pytest** (Python) **NUnit** (.NET), **Jest** (JavaScript).

3.

Automatización de pruebas unitarias.

- **Crear proyecto de prueba**: Agregar un proyecto de tipo “Prueba” y vincularlo al proyecto original.
- **Añades referencias**: Incluir el proyecto original y el framework de pruebas Microsoft.VisualStudio.QualityTools.UnitTestFramework.
- **Escribes pruebas**: Crear métodos [TestMethod] que verifiquen operaciones usando Assert.AreEqual.
- **Ejecutas y revisas**: Compilar y correr pruebas, detectar errores.
- **Correges y repetes**: Arreglar fallos y reejecutar hasta que todo pase.



3. Beneficios frente a pruebas manuales.

Se ejecutan solas y rápido, así puedes revisar muchas cosas sin estar pendiente.

Ayudan a detectar errores desde el principio y siempre funcionan igual, evitando fallos por distracciones.

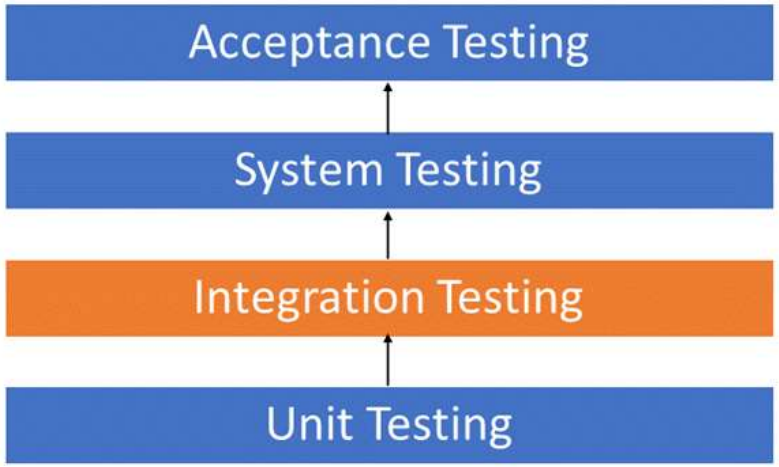
También facilitan cambiar o mejorar el código sin miedo a romper nada y sirven como guía de cómo debería funcionar cada parte.

Al principio llevan un poco de tiempo, pero a la larga ahorras mucho trabajo y problemas.

3.

Pruebas de integración
Qué son y cuándo se aplican.

Las **pruebas de integración** verifican que **varios módulos o componentes funcionen bien juntos**.



Se aplican cuando los elementos individuales ya funcionan y quieres asegurarte de que **el sistema completo interactúa correctamente**, como al conectar base de datos, servicios y funciones.

3.

Diferencias entre pruebas unitarias e integración.

	Pruebas Unitarias	Pruebas de Integración
Alcance	Una función o clase aislada	Varios módulos o componentes juntos
Objetivo	Detectar errores en el código individual	Detectar errores en la interacción entre componentes
Momento de uso	Durante el desarrollo de funciones	Después de que los módulos funcionan por separado
Velocidad	Rápidas y fáciles de automatizar	Más lentas, dependen de varias partes

3. Diseño y documentación de casos de prueba: importancia de registrarlos correctamente

El **diseño de casos de prueba** consiste en planificar cómo se va a probar cada función o módulo del software, definiendo qué funcionalidades se probarán, qué entradas se usarán, qué resultados se esperan y qué condiciones se deben cubrir, incluyendo casos normales, límites y errores.

Es igual de importante **documentar correctamente** estos casos, ya que permite reproducir las pruebas, verificar que todo funciona como se espera, facilitar el trabajo a otros testers o desarrolladores

4. Depuración de programas

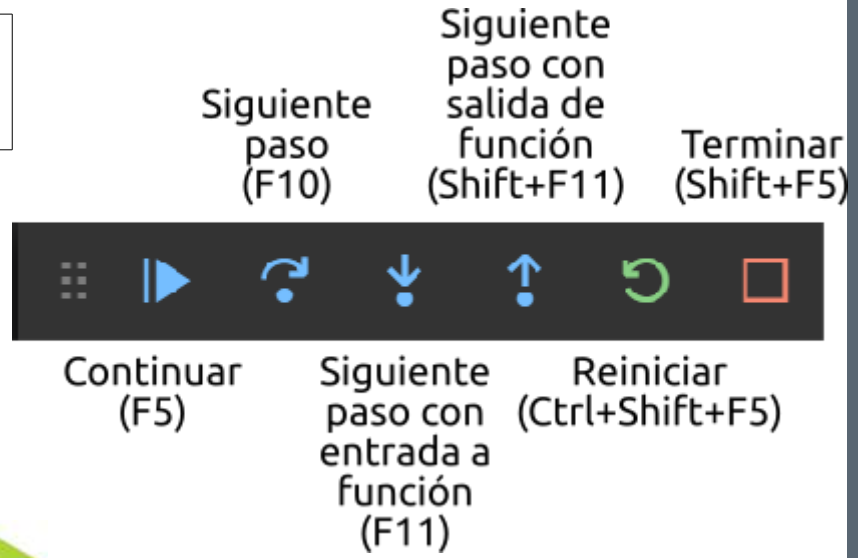
- ¿Qué es la depuración?
- Herramientas de depuración integradas en los entornos de desarrollo.
- Puntos de ruptura (breakpoints).
- Seguimiento en tiempo de ejecución.
- Examinadores de variables y su utilidad.

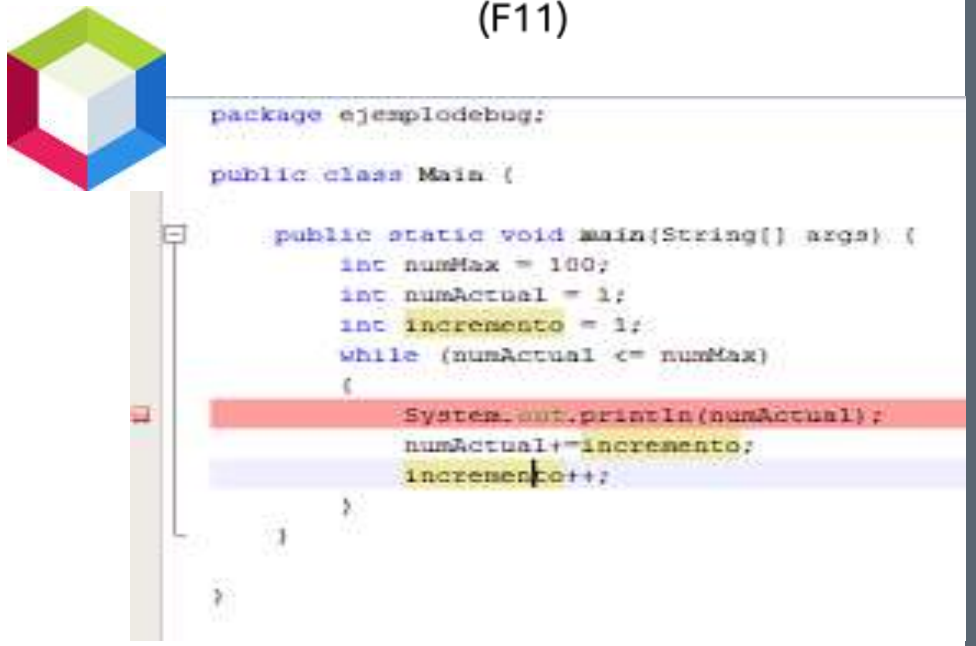
4.

¿Qué es la depuración?

La depuración es el proceso de encontrar, analizar y corregir errores en un programa

Se realiza con herramientas que permiten ejecutar el código paso a paso y observar las variables para detectar errores y mejorar la calidad del software

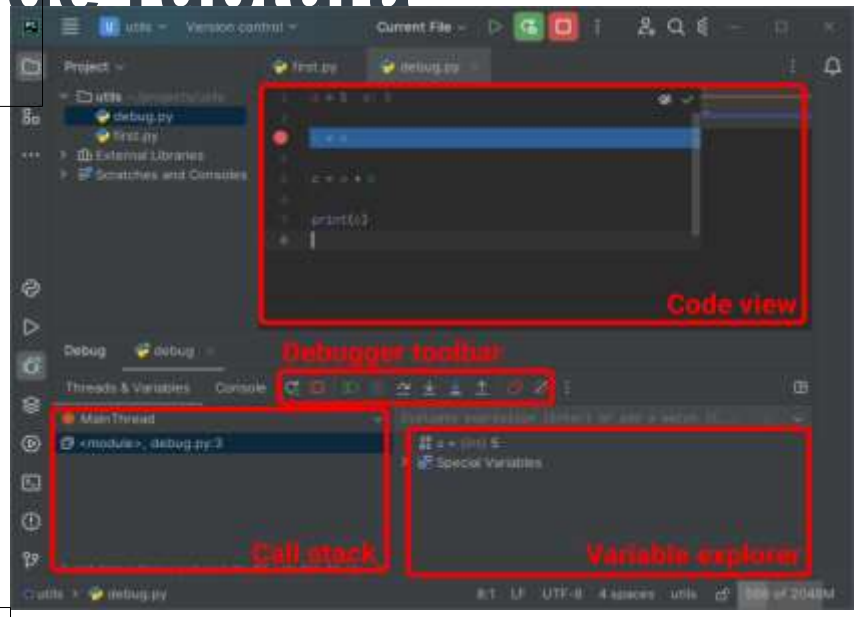


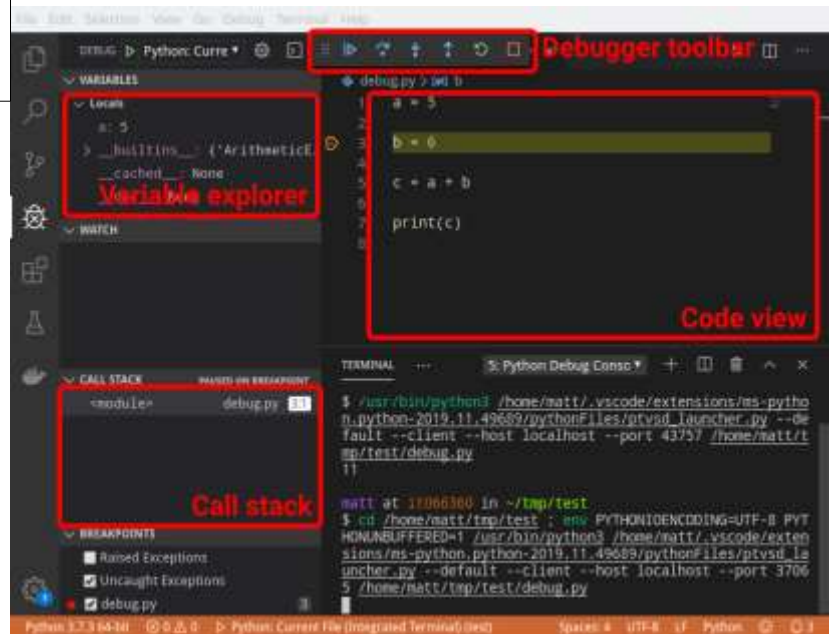


Herramientas de depuración integradas en los entornos de desarrollo y Puntos de ruptura

4.

- **Visual Studio Code:** depuración para múltiples lenguajes con breakpoints y ejecución paso a paso.
- **PyCharm:** depurador avanzado para Python con inspección de variables.
- **Chrome DevTools:** depuración de JavaScript



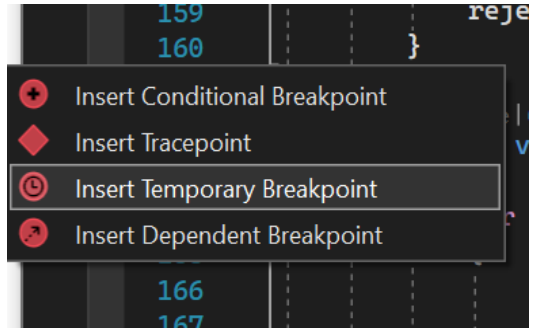


los entornos de desarrollo y Puntos de ruptura

4.

Un breakpoint es una herramienta de depuración que pausa la ejecución de un programa en un punto específico para inspeccionar variables, analizar el flujo del código y detectar errores...

Según la naturaleza existen 4 tipos:



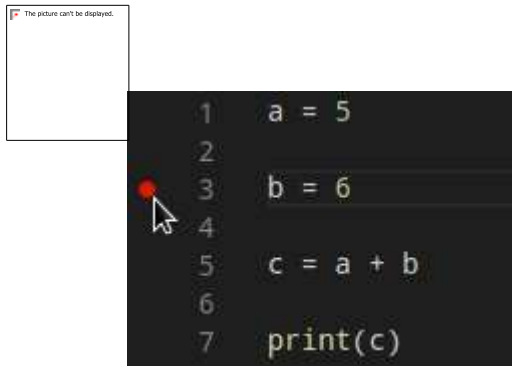
- **Estándar:** se activa al llegar a una línea.
- **Condicional:** se activa con una condición.
- **De excepción:** se activa cuando ocurre un error.
- **Por cambio de variable:** se activa cuando una variable cambia de valor.

Permite detener el programa en una línea concreta, revisar los valores en tiempo de ejecución, ejecutar el código paso a paso e identificar errores o comportamientos inesperados.

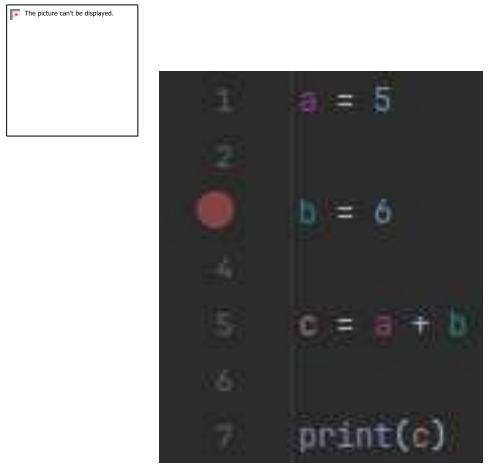
los entornos de desarrollo y Puntos de ruptura

4.

Resultan realmente útil si podemos indicarle al programa que **se detenga en ciertos puntos** para, por ejemplo, observar los valores de las variables.



La forma de indicarle al IDE que pause la ejecución en un punto concreto del programa es **colocando un breakpoint**.

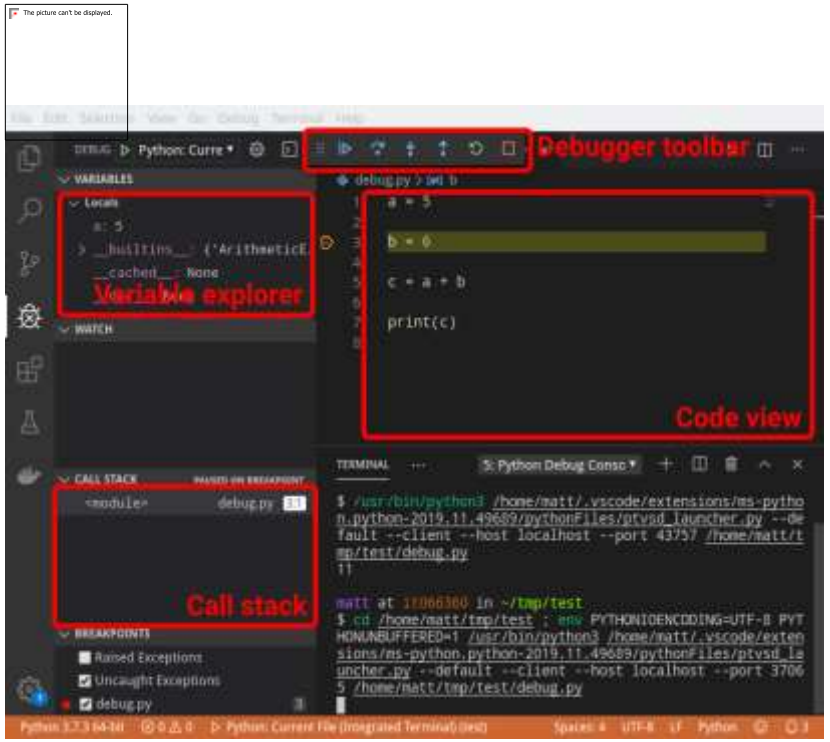


Para colocar un breakpoint, se hace clic en el **margen izquierdo (gutter)** del área de edición de texto, junto a los números de línea.

4. Seguimiento en tiempo de ejecución

El **tracing** (seguimiento) y el **debug** (depuración) permiten monitorear y corregir una aplicación mientras se ejecuta; esto incluye usar **breakpoints** (detener código) para inspeccionar el estado.

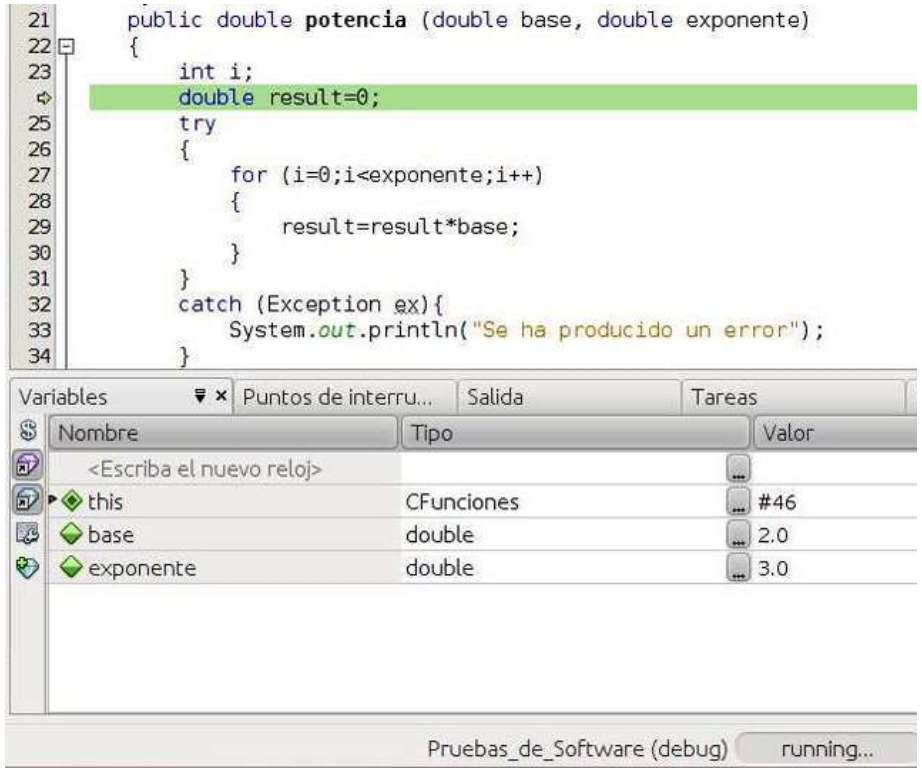
Hacer **stepping** (paso a paso) línea por línea o dentro de funciones, observar variables en **watch/scope** (vigilar variables), analizar la **call stack** (secuencia llamadas) que llevó a un estado determinado y registrar eventos y tiempos mediante **tracing/logging** (registro histórico) para evaluar rendimiento y comportamiento.



4. Examinadores de variables y su utilidad

Los **examinadores de variables** permiten seguir los valores y tipos de las variables durante la ejecución paso a paso de un programa, facilitando la detección de errores.

En **NetBeans**, la **Ventana de Inspección** muestra estos valores en tiempo real, ayudando a verificar que las funciones y bucles produzcan los resultados esperados.



5. Conclusión

- Resumen de ideas principales.

- Importancia de combinar pruebas y depuración para mejorar la calidad del software.

5.

CONCLUSION

Esta práctica nos ayudó a entender la importancia de las pruebas y la depuración para que el software funcione bien. Aprendimos sobre tipos de pruebas, automatización, casos de prueba y cómo usar herramientas para encontrar y corregir errores.

Combinar pruebas y depuración no es solo recomendable, sino elemental.

Mientras las pruebas identifican qué está mal, la depuración explica por qué y permite solucionarlo. Juntas forman un ciclo que garantiza un software más seguro.

Segunda Parte

Preguntas de tipo test

P

1.¿Qué objetivo principal tienen las pruebas unitarias?

a) Verificar que varios módulos funcionen correctamente juntos.

b) Detectar errores en el código individual, como funciones o clases aisladas.

c) Cambiar o mejorar el código automáticamente

d) Documentar el software sin ejecutar código.

2.¿Cuál de los siguientes tipos de breakpoint se activa únicamente cuando **una variable específica cambia su valor en tiempo de ejecución?**

a) Estándar

b) Condicional

c) De excepción

P

3.¿Qué describe mejor las pruebas de integración?

a) Verificar el comportamiento de funciones o métodos individuales.

b) Detectar errores en la interacción entre varios módulos o componentes del sistema

c) Ejecutar pruebas sin necesidad de automatización.

d) Documentar casos de prueba sin comprobar resultados.

4. ¿Cuál es el objetivo principal de la técnica de Clases de Equivalencia?

a) Probar absolutamente todos los valores posibles de entrada.

b) Reducir el número de casos de prueba agrupando datos con comportamiento similar.

c) Probar únicamente los valores extremos de los rangos.

d) Acceder al código fuente para corregir errores de sintaxis.

P.

5. ¿Cuál es la diferencia principal entre Verificación y Validación en el contexto de las pruebas?

a) La verificación busca errores de código, la validación busca errores de diseño.

b) La verificación asegura que el producto se construye correctamente, la validación asegura que es el producto correcto para el usuario.

c) No hay diferencia, son sinónimos.

d) La validación se hace antes de escribir el código y la verificación después.